

Hackathon Mentor Handout

Up: [Map](#) 

Hackathon Mentor Handout

Tags: [#MomTest](#) [#Hackathon](#) [#UserResearch](#) [#ProductValidation](#) [#VibeCoding](#)
[#Fintech](#)

Notes

Highlights

Step-by-step idea-validation guide for hackathon participants who can't get real users on a 48-hour clock. Six thinking-tool tests instead of user interviews - what's the actual job, is this previously impossible, where's the moat, what's the killer demo. Designed for fintech hackathon teams but works for any 48-hour build

Read This First (60 seconds)

You've got an idea. You've got 48 hours. You're tempted to start coding right now.

Don't. The teams who win hackathons spend the first 2-3 hours making sure their idea isn't going to embarrass them on Sunday. The teams who lose spend hour 47 still trying to make their auth flow work.

This guide is six thinking-tool tests. The whole thing takes 90 minutes. You don't need to talk to real users (you probably can't on a hackathon clock) - you just need to honestly pressure-test your own idea using frameworks that VCs, product leaders, and AI experts use to spot bad ideas before they're built.

Most of these tests come from people much smarter than me - Benedict Evans (a16z, ex-Andreessen), Andrej Karpathy (founding OpenAI, ex-Tesla AI), Max Junestrand (Legora CEO), Jensen Huang (Nvidia). The "Mom Test" name comes from Rob Fitzpatrick's book - the original idea was that even your mum will lie to you about whether your business idea is good. So you need a way to get to the truth instead of the polite answer.

Skip a test at your own risk.

Step 1 - Write Your Idea In One Sentence (10 minutes)

Get the team to agree on this exact sentence:

"We're building [a thing] that helps [a specific person] do [a specific task] better than [what they currently do]."

Examples:

- "We're building a tool that helps junior credit analysts at hedge funds spot antitrust cases worth funding, faster than reading EU Commission PDFs by hand."
- "We're building an app that helps freelance bookkeepers in the UK file their clients' VAT returns, faster than copying numbers between spreadsheets."

If you can't fill in all four blanks specifically (with one role, one task, one alternative), you don't have an idea yet - you have a vague vibe. Stop and sharpen.

Common traps:

- "Helps small businesses..." - too broad. Which small businesses? The 5-person dental practice or the 50-person construction firm?
- "Helps people manage their finances..." - what specific moment? Filing tax? Reconciling expenses? Choosing a pension?
- "Powered by AI..." - that's not the idea, that's the engine. What's the *thing it does*?

Don't move to Step 2 until the whole team can recite the sentence from memory.

Step 2 - The Job-To-Be-Done Test (15 minutes)

This is from Benedict Evans.. Every product splits into two layers:

1. **The actual thing the customer is paying for** (the job to be done)
2. **The way you currently deliver it** (the process, the tech, the AI)

Most teams confuse these. They build something cool with AI and assume the AI IS the product. It isn't. The product is the *job* it does for someone.

Three questions for your idea:

1. **What is the customer ACTUALLY paying for?** (Not "an AI tool that..." - finish the sentence with the outcome they want.)
2. **What were they paying for BEFORE this was technically possible?** (If your answer is "nothing" - they weren't doing this at all - that's risky. Usually means it isn't a real job. If your answer is "they paid junior associates / consultants / spreadsheets / their own evenings" - good. That's the budget you're competing for.)
3. **If your AI got 10x better tomorrow, would your buyer pay 10x more?** (If yes, the AI is the product. If no, the AI is the delivery mechanism and something else is

the product.)

Worked example - bookkeeper VAT app:

- Bad framing: "AI that auto-categorises bank transactions for VAT."
- Real job framing: "Removes the panic week before quarterly VAT deadlines for sole-trader bookkeepers running 30+ small clients."
- Before AI, they paid: temp staff, late-night work, accountant subcontractors. Real budget. Real job.
- If your tool got 10x better, would they pay 10x more? Probably not - the budget is capped by what they used to pay temps. Tells you the price point.

If you can't answer all three honestly, you haven't found the job yet.

Step 3 - The "Previously Impossible" Test (15 minutes)

This is from Andrej Karpathy (founding OpenAI, ex-Tesla AI):

1. **The same thing, cheaper** (replace a junior with a bot)
2. **More of the same thing, for the same price** (do 10x the work)
3. **Something that was previously impossible** (a thing nobody could do at any headcount)

The first two are commodity. Anyone can replicate them in 6 months. The third is where moats live.

Ask: **what does our product let someone DO that they literally couldn't do before, at any cost?**

Worked example - antitrust litigation tool:

- Cheaper: "Read EU Commission decisions faster than a junior associate." OK, but a £200K Burford analyst could already do this. Junior staff get cheaper every year. Weak moat.
- More of the same: "Screen 100 cartel cases a week instead of 5." Better. Funders genuinely couldn't afford the headcount to do this.
- **Previously impossible:** "Score every cartel decision the EU has ever published, daily, against your fund's specific risk criteria, and proactively alert you to follow-on opportunities before any other funder spots them." This is not a faster typewriter - it's a new capability. No human team could ever do this.

The "previously impossible" version is what you want to demo on Sunday. It's also the version that makes the pitch unforgettable.

If your idea sits entirely in category 1 or 2, you're building a faster-typewriter business. They exist, but at a hackathon you'll get crushed by teams demoing magic.

Step 4 - The Scaffolding Beats Model Test (15 minutes)

This is from Max Junestrand, CEO of Legora (legal AI company that scaled from 50 to 750 law firm clients in 12 months, valued at \$5.5B). His key line:

"80% comes from building normal software around the models. All the scaffolding, all the ways that the users can actually interact with the system. The model is 20%."

Translation: the AI prompt is NOT the product. The product is everything around it - the workflow, the data you feed it, the way you display the answer, the integrations. Anyone with an OpenAI API key can copy your prompt in a weekend. They can't copy a year of domain expertise.

Two questions:

1. **What's the 80% scaffolding for our idea?** (The workflow, the data sources, the format of the output, the integrations, the trust mechanisms. Not the AI call.)
2. **What can our team build in this 80% that someone else can't replicate in a weekend?** (Domain knowledge from one team member who worked in the industry? A unique dataset? A specific integration? A workflow you've watched someone do for 3 years?)

If your answer to question 2 is "we know how to write a good prompt", you've got nothing. Everyone has prompts.

Worked example - antitrust tool:

- Bad: "We use Claude to summarise EU cartel decisions." (Anyone can do this.)
- Good: "We've built a curated database of every EU cartel case decided since 2010, tagged by industry, market, overcharge percentage, and follow-on outcome. We feed that to Claude as context for every new decision the system parses. The model is interchangeable - the database is the moat." (Now someone has to recreate 15 years of curation to compete.)

The hackathon test: **what's the one piece of domain expertise on your team that the team next to you doesn't have?** Build around that. If the answer is "nothing", borrow some - find a mentor or a friend in the domain and use them as your scaffolding.

Step 5 - The Radiology Paradox (10 minutes)

Ask: **does our product replace an expert, or does it capture their judgment and make it portable?**

The first version (replacement) is hostile to your buyer. They will fight you. The second version (capture-and-scale) is something the expert WANTS, because it makes them 10x more productive instead of unemployed.

Worked example - bookkeeper VAT app:

- Replacement framing: "AI that does VAT returns so you don't need a bookkeeper." (Bookkeepers will fight you. Bad strategy.)
- Capture framing: "Tool that captures how YOU categorise transactions, learns your firm's specific rules, and applies them to every new client so you can take on 3x more accounts." (Bookkeepers will champion you. Better strategy.)

Same product, completely different positioning. The second one wins.

Step 6 - The Pre-Mortem (15 minutes)

Imagine it's Sunday afternoon. You've just bombed your demo. The judges look bored. The team next to you got an investor introduction, you got a polite nod.

Why did it fail? Write down the top 5 reasons.

Common pre-mortem failures:

- The demo was too complex - judges didn't understand what they were looking at in 90 seconds
- The "magic moment" wasn't actually magical - it just felt like a slow ChatGPT
- The AI hallucinated on stage and the team had to apologise
- The team built the boring stuff (auth, settings, dashboard) and ran out of time for the wow moment
- The data was lorem ipsum and the demo felt like a toy
- The pitch led with "the market is \$50B" instead of one specific user's specific pain
- Someone in the audience had built something similar 6 months ago

Each of these has a fix - and the fix is cheaper to apply BEFORE you build than after. Look at your top 5. For each one, write down what you'd change in your plan today to prevent it.

This 15-minute exercise is the single highest-ROI thing in the whole guide. The teams who win have already imagined the failure and walked it back.

Step 7 - Phone The Smartest Person You Know (Optional, 20 minutes)

You probably can't talk to real users in a hackathon weekend. But you can talk to ONE person who knows the domain.

Find one person on your team's network - an old boss, a friend's parent, a LinkedIn contact, a mentor at the hackathon - who has worked in the industry your idea is for. Send them this exact message:

"Hi - I'm at [hackathon] this weekend building something for [the kind of person]. Can I steal 10 minutes to ask you to poke holes in the idea? You won't hurt my feelings - the more brutal the better."

When they call back, don't pitch. Ask:

1. "Imagine someone built this. What's the first reason you'd guess it would fail?"
2. "Who's already tried this? What happened?"
3. "If you were in this role, would you actually use this? Why or why not?"
4. "What am I missing about how this industry actually works?"

Take notes. Their objections are gold.

If you've got a mentor at the hackathon (hi), they count. Use them.

The Scorecard (5 minutes)

After the six tests, score honestly. 1 point per row. Be brutal.

#	Test	Pass = 1 if...
1	One-sentence idea works	All four blanks filled in specifically. The whole team can recite it from memory
2	JTBD - real job named	You can name what someone WAS paying for before this was possible (consultants, juniors, their evenings, etc.)
3	JTBD - AI is delivery, not product	You're clear what the customer is actually paying for, separate from the AI that delivers it
4	Previously-impossible angle exists	You've identified ONE thing your tool will do that nobody could do before at any headcount
5	Domain scaffolding identified	You can name the 80% that isn't the AI - the data, the workflow, the integration that gives you a moat

#	Test	Pass = 1 if...
6	Team has one piece of domain expertise	At least one team member or mentor brings industry-specific knowledge that the next team doesn't have
7	Capture-not-replace positioning	Your pitch makes the expert want to use you, not fear you
8	Pre-mortem done, fixes applied	You've imagined Sunday's failure and changed your plan accordingly
9	One-sentence demo plan	You can finish: "On Sunday we'll demo a [user] doing [action] and getting [result]"
10	You've decided what NOT to build	You've named features you're explicitly NOT shipping. If your list is empty, you're going to over-build

Total: ____ / 10

- **8-10:** Strong. Start building immediately.
- **6-7:** Good enough. Build, but assign one team member to revisit the weak tests on Saturday morning.
- **4-5:** Wobbly. Fix the gaps before writing code. Often a 30-min team conversation does it.
- **0-3:** Stop. The idea isn't ready. Either pick a different one or spend an hour with a mentor reframing.

Decide What To Build (30 minutes)

The biggest mistake at hackathons is over-scoping. Here's how to shrink it.

The smallest viable demo

Pick **one** moment of magic from your idea. The single thing that, when a judge sees it, makes them go "huh, that's clever."

Build THAT moment end-to-end with real data. Mock or stub everything else.

Examples:

- **Antitrust litigation tool:** Don't build the whole funder workflow. Demo: "Here's a real EU Commission cartel decision PDF [drag in]. Tool reads it, identifies the affected industries, estimates damages range. Output in 30 seconds."
- **Bookkeeper VAT app:** Don't build the whole accounting suite. Demo: "Here's a real bank statement [paste]. Tool categorises every transaction for VAT and

produces a filing-ready summary."

The mock parts are FINE. Judges expect them. Make the magic moment work flawlessly.

Cut everything that's not on screen Sunday

Things you don't need:

- Login / signup flow (skip - assume the user is in)
- Settings page (don't build)
- Admin dashboard (definitely not)
- Multi-user support (no)
- Pretty error messages (no - happy path only)
- Mobile responsive (probably no - demo on the laptop you'll be using)
- Database persistence (often no - in-memory state is fine for a demo)

If it's not visible during the 3-minute pitch, it doesn't exist.

Vibe-code the boring stuff

Use Claude Code, Cursor, v0, Lovable, or whatever AI tool you're comfortable with for:

- The UI scaffolding
- The API plumbing
- The deployment
- The boilerplate

Spend your human hours on:

- The prompt that does the magic (this is the actual product)
- The demo (rehearse it 5 times)
- One real working slice (not five half-broken slices)

If you're a mentor and the team needs help with prompts or vibe-coding, grab a Claude expert (every hackathon has one).

Plan The Demo (Saturday Night, 30 minutes)

Most demos are: "We're a team of X. The market is Y billion. Here are our features..."
Boring. Forgettable.

Win-shaped demos look like this:

1. **30 seconds: the user, the moment, the pain.** A specific person, a specific Tuesday, a specific frustration. "Meet Sarah. She's a credit analyst at a litigation funder. Last Tuesday she spent 6 hours reading a 200-page EU cartel decision to figure out if it was worth a £2M investment."
2. **30 seconds: what they do today.** "Today, every funder analyst does this manually. £200K of due diligence per case. 3-month decision cycles. They miss most cases that come up."
3. **90 seconds: live demo.** Real input. Real output. The magic moment. NO SLIDES. Just the working product.
4. **30 seconds: the wedge + roadmap.** "What you saw is v1. Next: monitoring, multi-jurisdictional, supply-chain mapping. We start with [specific buyer] who pays [specific price]."

Practice it 5 times before you pitch. Time yourself. If you're over 3 minutes, cut.

The Friend-Bias Check

Most hackathon validation comes from teammates, mentors, or friends who want you to feel good. Their feedback is contaminated. Signs your validation was honest, not polite:

- Someone corrected something you said (downgraded volume, killed a feature, pushed back on the price)
- Someone volunteered specific failures or admissions of weakness from their own work
- Someone gave you a constraint you didn't ask for ("would only work if X")
- Someone said the magic phrase: "have you thought about Y?" (means they're engaged, not just polite)

If everyone you talked to nodded along and said "great idea," you didn't validate. You got encouraged. Re-do Step 6 (the pre-mortem) - that's the cheapest way to simulate hostile feedback when you can't get it from outside.

Tags

#MomTest , #Hackathon , #UserResearch , #ProductValidation , #VibeCoding , #Fintech